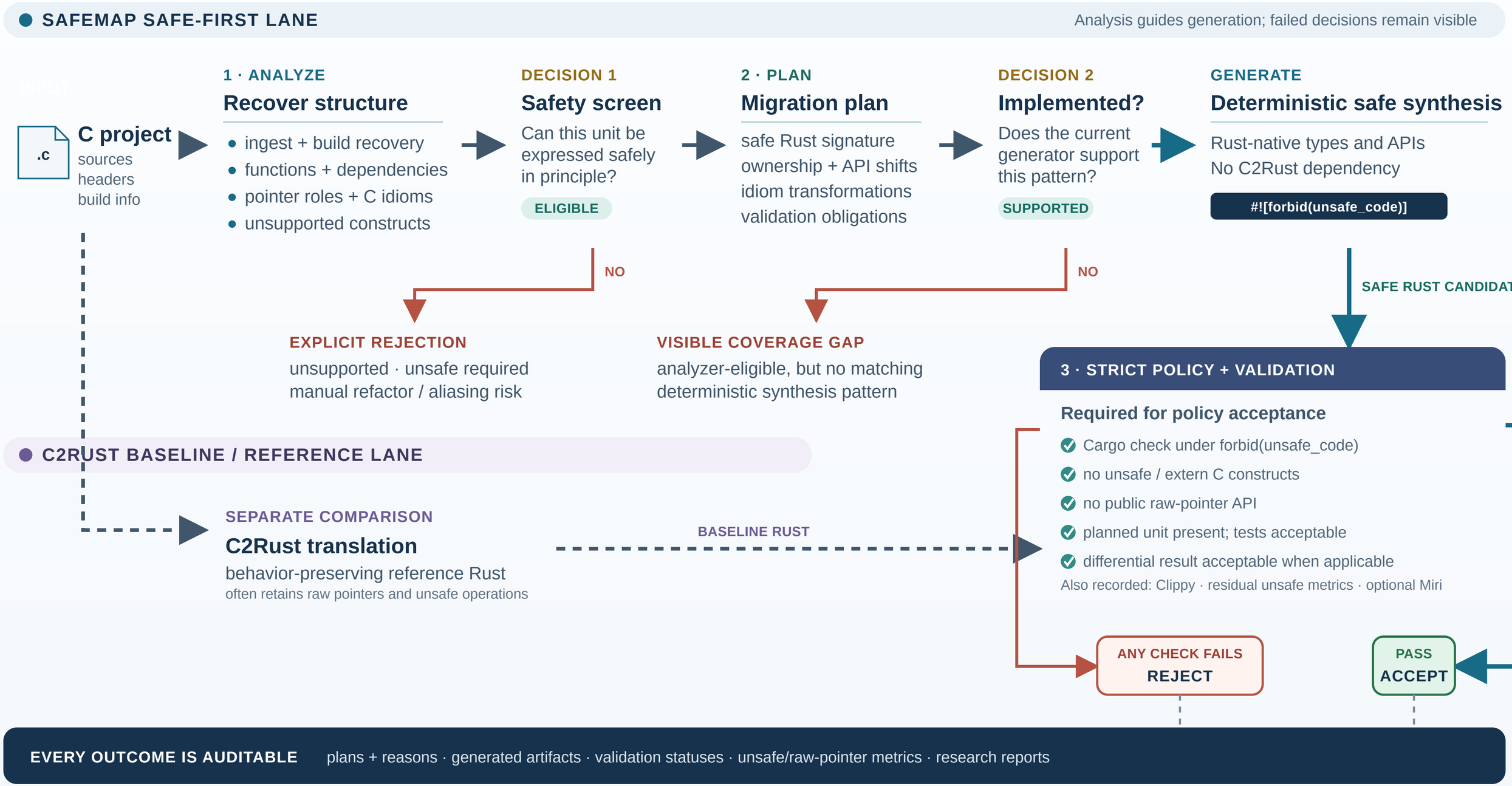
SAFEMAP SAFE-FIRST LANE
Analysis guides generation; failed decisions remain visible
INPUT
.c
C project
sources
headers
build info
1 · ANALYZE
Recover structure
ingest + build recovery
functions + dependencies
pointer roles + C idioms
unsupported constructs
DECISION 1
Safety screen
Can this unit be expressed safely in principle?
ELIGIBLE
NO
EXPLICIT REJECTION
unsupported · unsafe required
manual refactor / aliasing risk
2 · PLAN
Migration plan
safe Rust signature
ownership + API shifts
idiom transformations
validation obligations
DECISION 2
Implemented?
Does the current generator support this pattern?
SUPPORTED
NO
VISIBLE COVERAGE GAP
analyzer-eligible, but no matching deterministic synthesis pattern
GENERATE
Deterministic safe synthesis
Rust-native types and APIs
No C2Rust dependency
#![forbid(unsafe_code)]
SAFE RUST CANDIDATE
3 · STRICT POLICY + VALIDATION
Required for policy acceptance
Cargo check under forbid(unsafe_code)
no unsafe / extern C constructs
no public raw-pointer API
planned unit present; tests acceptable
differential result acceptable when applicable
Also recorded: Clippy · residual unsafe metrics · optional Miri
ANY CHECK FAILS
REJECT
PASS
ACCEPT
C2RUST BASELINE / REFERENCE LANE
SEPARATE COMPARISON
C2Rust translation
behavior-preserving reference Rust
often retains raw pointers and unsafe operations
BASELINE RUST
EVERY OUTCOME IS AUDITABLE
plans + reasons · generated artifacts · validation statuses · unsafe/raw-pointer metrics · research reports